ECE 726 Project Proposal

Presentation on Transformers

Department of Electrical and Computer Engineering, McMaster University
Machine Learning: An Introduction – ECE 726

Mahan Choudhury, Student ID: 400648097

April 15, 2026

Contents

1 Introduction

Sequential modelling has historically relied on recurrent architectures such as Long Short-Term Memory (LSTM) networks, which process tokens one time-step at a time. This inherently sequential computation prevents parallelization during training and makes it difficult to model long-range dependencies, since gradient information travel through many recurrent steps.

Vaswani et al. introduced the *Transformer*, a model that dispenses with recurrence entirely and relies instead on *self-attention* to relate every position in a sequence to every other position in a single, parallel operation [1]. Observing its foundational impact on modern machine learning, I intend to explore and present on this architecture for my ECE 726 course project.

This proposal outlines a study of the Transformer architecture centred on the following core components:

(i) Input embedding

(ii) Positional encoding

(iii) Scaled dot-product attention and multi-head attention (Query, Key, Value)

(iv) Layer normalisation and position-wise feed-forward sub-layers

(v) Encoder and Decoder structures

The primary study material is the original Vaswani et al. paper [1] and Chapter 12 of Bishop [2]

Notation. Throughout this proposal, K denotes vocabulary size, D denotes the model embedding dimensionality (equivalent to d_{model} in Vaswani et al.), N denotes the number of encoder or decoder layers, and n indexes sequence positions.

2 Mathematical Formulation

2.1 Input Embedding

Let the input sequence consist of tokens represented as one-hot encoded vectors x_n, where K is the dimensionality of the vocabulary dictionary. To avoid the high dimensionality and sparsity of one-hot representations, each token is mapped into a continuous, lower-dimensional embedding space of dimensionality D (equivalent to the Transformer's d_{model}). The embedding process is defined by a learned matrix E of size $D \times K$. For each one-hot encoded input vector x_n, the corresponding embedding vector v_n is calculated as:

$$v_n = Ex_n \tag{1}$$

Because x_n utilizes a one-hot encoding, the resulting vector v_n is simply given by the corresponding column of the embedding matrix E.

As Bishop notes, mapping words into this dense continuous space allows the network to capture elementary semantic properties, mapping words with similar meanings to nearby locations in the embedding space [2, Sec. 12.1]. Following Vaswani et al., this same weight matrix is shared between the encoder embedding layer, the decoder embedding layer, and the pre-softmax linear transformation [1, Sec. 3.4]. Before being added to the positional encodings, these embedded vectors are scaled by $\sqrt{D}$.

2.2 Positional Encoding

Because self-attention is permutation-invariant by construction, positional information must be injected explicitly into the representation. Vaswani et al. propose the following fixed sinusoidal encoding [1, Sec. 3.5]:

$$\mathrm{PE}(n,\, 2i) = \sin\!\left(\frac{n}{10000^{\,2i/D}}\right), \tag{2}$$

$$\mathrm{PE}(n,\, 2i+1) = \cos\!\left(\frac{n}{10000^{\,2i/D}}\right), \tag{3}$$

where n is the position index and $i \in \{0, \ldots, D/2 - 1\}$ is the dimension index. The scaled embedding vector is then combined with the positional encoding to produce the final input representation [1, Sec. 3.5]:

$$\tilde{v}_n = \sqrt{D}\; v_n + \mathrm{PE}(n, \cdot). \tag{4}$$

Bishop provides additional intuition, stating that the sinusoidal form allows $\mathrm{PE}(n+k)$ to be expressed as a linear function of $\mathrm{PE}(n)$ for any fixed offset k [2, Sec. 12.1.9].

2.3 Scaled Dot-Product Attention

Given a query matrix $Q \in \mathbb{R}^{T_q \times d_k}$, a key matrix $K_{\mathrm{mat}} \in \mathbb{R}^{T_k \times d_k}$, and a value matrix $V_{\mathrm{mat}} \in \mathbb{R}^{T_k \times d_v}$, the scaled dot-product attention is defined as [1, Eq. (1), Sec. 3.2.1]:

$$\mathrm{Attention}(Q,\, K_{\mathrm{mat}},\, V_{\mathrm{mat}}) = \mathrm{softmax}\!\left(\frac{Q\, K_{\mathrm{mat}}^{\top}}{\sqrt{d_k}}\right) V_{\mathrm{mat}}. \tag{5}$$

The $\sqrt{d_k}$ scaling factor prevents the dot products from growing large in high-dimensional spaces, which would push the softmax into regions of extremely small gradients [1, Sec. 3.2.1]. Each row of the output is a convex combination of the value vectors, weighted by the compatibility of the corresponding query with each key.

Masked self-attention (decoder). To preserve the autoregressive property in the decoder, future positions must be hidden. This is achieved by adding a mask M_{mask} to the pre-softmax logits [1, Sec. 3.2.3], which ensures the softmax assigns zero weight to all future positions.

2.4 Multi-Head Attention

Rather than applying a single attention function over the full D-dimensional space, multi-head attention projects queries, keys, and values h times with distinct learned linear projections and computes attention in parallel across h lower-dimensional subspaces [1, Sec. 3.2.2]:

$$H_i = \text{Attention}\Big(Q\,W_i^Q,\ K_{\text{mat}}\,W_i^K,\ V_{\text{mat}}\,W_i^V\Big), \tag{6}$$

$$\text{MHA}(Q,\ K_{\text{mat}},\ V_{\text{mat}}) = \text{Concat}(H_1, \ldots, H_h)\,W^O, \tag{7}$$

with projection matrices $W_i^Q \in \mathbb{R}^{D \times d_k}$, $W_i^K \in \mathbb{R}^{D \times d_k}$, $W_i^V \in \mathbb{R}^{D \times d_v}$, and $W^O \in \mathbb{R}^{hd_v \times D}$.

The base model uses $h = 8$ heads and $D = 512$, giving $d_k = d_v = D/h = 64$ [1, Sec. 3.2.2].

2.5 Position-Wise Feed-Forward Network

After the attention sub-layer, each token representation is passed through an identical two-layer fully connected network applied independently at each position [1, Sec. 3.3]:

$$\text{FFN}(z) = \max(0,\ z\,W_1 + b_1)\,W_2 + b_2, \tag{8}$$

with $W_1 \in \mathbb{R}^{D \times d_{ff}}$, $W_2 \in \mathbb{R}^{d_{ff} \times D}$, and inner dimensionality $d_{ff} = 2048$ in the base model.

2.6 Residual Connections and Layer Normalisation

Each sub-layer (attention or FFN) is wrapped with a residual connection followed by layer normalisation. The combined operation for a generic sub-layer is [1, Sec. 3.1]:

$$z' = \text{LayerNorm}(z + \text{SubLayer}(z))\,. \tag{9}$$

2.7 Encoder

The encoder maps the embedded input sequence $\tilde{V} = [\tilde{v}_1, \ldots, \tilde{v}_T]$ to a sequence of contextualised representations. It is composed of $N = 6$ identical layers, each containing two sub-layers: multi-head self-attention followed by a position-wise FFN. Setting $Z^{(0)} = \tilde{V}$, for encoder layer ℓ:

$$A^{(\ell)} = \text{LayerNorm}\big(Z^{(\ell-1)} + \text{MHA}\big(Z^{(\ell-1)},\ Z^{(\ell-1)},\ Z^{(\ell-1)}\big)\big)\,, \tag{10}$$

$$Z^{(\ell)} = \text{LayerNorm}\big(A^{(\ell)} + \text{FFN}\big(A^{(\ell)}\big)\big)\,. \tag{11}$$

2.8 Decoder

The decoder also has $N = 6$ layers, but each layer contains three sub-layers [1, Sec. 3.1]:

(1) **Masked multi-head self-attention** over the decoder's own shifted output to prevent attending to future positions.

(2) **Multi-head cross-attention**, where queries come from the previous decoder sub-layer output $S^{(\ell)}$ and keys and values come from the final encoder output $Z^{(N)}$:

$$C^{(\ell)} = \mathrm{LayerNorm}\big(S^{(\ell)} + \mathrm{MHA}\big(S^{(\ell)},\ Z^{(N)},\ Z^{(N)}\big)\big)\,. \tag{12}$$

(3) **Position-wise FFN** with the same residual-and-norm pattern as the encoder.

Output projection and training loss. The final decoder representation is projected to a probability distribution over the vocabulary using the transposed embedding matrix $E^\top$, consistent with the shared weight convention of Eq. (1):

$$p_n = \mathrm{softmax}\big(E^\top\, z_n^{(N)}\big)\,, \tag{13}$$

where the components of $z_n^{(N)}$ are defined as follows:

- z: Represents the continuous, D-dimensional vector representation of the token.
- n: Represents the specific position or time-step in the sequence currently being decoded.
- (N): Represents the layer index. Since the decoder is composed of a stack of N layers (where $N = 6$ in the base model), the superscript (N) indicates that this is the output from the very last layer of the decoder stack.

Training minimises the per-token cross-entropy loss summed over the target sequence.

3 Transformer Architecture Diagram

Figure 1 shows the full encoder–decoder Transformer following Vaswani et al. [1, Fig. 1].

4 Papers and Study Material

This project draws exclusively from two sources:

1. A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention Is All You Need,"
2. C. M. Bishop, *Deep Learning: Foundations and Concepts*, Springer, 2024, Chapter 12.

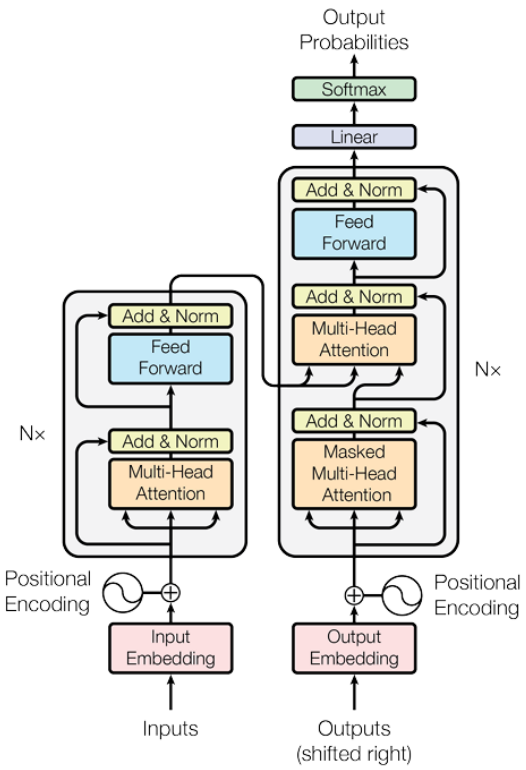

Figure 1: Encoder-decoder Transformer architecture

5 Summary

The Transformer replaces recurrence with self-attention and enables full parallelization during training and constant-depth access to long-range dependencies. This proposal outlines a study plan of the architecture through its core mathematical components; such as input embeddings (E, v_n), sinusoidal positional encoding, scaled dot-product and multi-head attention, position-wise feed-forward sub-layers, layer normalization, and the encoder–decoder framework; using consistent notation throughout (K for vocabulary size, D for model dimensionality, n for sequence position).

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[2] C. M. Bishop, *Deep Learning: Foundations and Concepts*. Springer, 2024. [Online]. Available: https://www.bishopbook.com/